

Introduction

Des outils personnalisés aux fonctionnalités intégrées

Votre parcours :

- **Partie 1** : compréhension approfondie des outils et création d'outils de fonction personnalisés
- **Partie 2** : utilisation des outils prêts pour la production fournis par ADK

Ce qui change : au lieu d'écrire du code d'implémentation, vous importez et utilisez des outils prédéfinis dont ADK assure la maintenance.

Problème

Réinventer les capacités courantes

Supposons que vous souhaitiez ajouter la recherche sur le Web à votre agent de la première partie :

```
# Comment faire pour implémenter la recherche sur le Web ?
def search_web(query: str) -> dict:
    """Recherche des informations sur le Web."""
    # Concrètement, comment se passe l'implémentation ?
    # - Nécessité d'appeler une API de recherche (laquelle et comment
    s'authentifier ?)
    # - Analyse et mise en forme des résultats pour le LLM
    # - Gestion des limitations du débit et des exceptions
    # - Maintien à jour de l'intégration d'API
    # - Optimisation des résultats pour le LLM
    # Ce n'est pas chose facile !
    pass
```

Problèmes rencontrés en créant tout soi-même :

-  **Implémentation complexe** : intégration d'API, authentification, gestion des exceptions
-  **Charge de maintenance** : modifications, mises à jour et abandons d'API
-  **Pas d'optimisation** : résultats à mettre en forme pour que le LLM les utilise
-  **Fonctionnalités de production manquantes** : limitation du débit, mise en cache et surveillance
-  **Perte de temps** : des heures, voire des jours, pour créer ce qui devrait prendre quelques minutes

Problème fondamental : des capacités courantes comme la recherche sur le Web et l'exécution de code nécessitent de gros efforts d'ingénierie pour être correctement implémentées.

